

Cache Poisoning at Scale

Cache Poisoning at Scale - @iustinBB, youst.in.

- Published: date not stated
- Original: <<https://youst.in/posts/cache-poisoning-at-scale/>>
- Preserved from: <https://youst.in/posts/cache-poisoning-at-scale/> (live) on 2026-08-09
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Web Hacking Techniques Index collections, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Even though Web Cache Poisoning has been around for years, the increasing complexity in technology stacks constantly introduces unexpected behaviour which can be abused to achieve novel cache poisoning attacks. In this paper I will present the techniques I used to report over 70 cache poisoning vulnerabilities to various Bug Bounty programs. If you aren't already familiar with the basics of Web Cache Poisoning, I highly recommend you read [Practical Web Cache Poisoning](#) by albinowax.

Backstory

On December 19, 2020 I published a short write-up on a particular edge-case affecting Varnish configurations, where sending a capitalized host header could have poisoned the cache. Unfortunately, since this required a particular custom Varnish configuration, scanning for it netted me no other results. To my surprise, shortly after publishing the write-up I realized Cloudflare was also vulnerable to the same capitalized host header attack, but this time, it required no custom configuration. This meant cloudflare lowercased the host header before introducing it into the cache key, but always forwarded as sent by the client. If any backend behind Cloudflare would respond with a different response when sent a capitalized host header, it would allow the cache to be poisoned. You can read more about this specific technique in my [previous write-up](#), however both Fastly and Cloudflare have now fixed the behaviour. Since this subtle inconsistency affected a good subset of bug bounty targets I decided to see what other common patterns I could identify and exploit at scale.

Incorrect Handling of the URL Fragment in Apache Traffic Server ([CVE-2021-27577](#))

Apache Traffic Server (ATS) is a caching HTTP proxy used widely by Yahoo and Apple. When a request sent to ATS contains a url fragment, ATS forwards it without stripping the fragment. According to [RFC7230](#), the requests forwarded by ATS are invalid, since the origin-form should only be composed of the absolute-path and query.

!

Moreover, ATS generates cache keys by extracting the host, path and query, ignoring the url fragment. This therefore means that both requests below will share the same cache key:

!!

ATS ignoring the url fragment when generating the cache key, but still forwarding it creates a huge opportunity for cache poisoning attacks. When the proxies behind ATS are configured to encode `#` to `%23`, it makes it possible for an attacker to cache a completely different path under any cache key. I was able to use this technique to poison static files like so:

!

If the backend also normalized `/../`, it would allow an attacker to redirect users to any path, allowing for easy escalation for XSS and Open redirects.

!

GitHub CP-DoS

Since a big part of Cache Poisoning vulnerabilities are caused by unkeyed headers, I wrote a tool that would bruteforce unkeyed headers and detect Cache Poisoning. This allowed me to quickly scan bug bounty targets at scale.

Since a lot of bug bounty programs include Github Repositories in their scope list, a few repo urls made it into to my targets list. While going through the scan results, I noticed all github repositories were being marked as vulnerable to Cache Poisoning when the header `content-type` contained an invalid value.

!

Even though the scan was marking Github Repos as vulnerable and the attack worked in Burpsuite, I was unable to replicate in a browser. It quickly became apparent that Github was including the Authentication cookie inside the cache key. While it was not possible to poison repos for authenticated users, it was possible to take down repositories for all unauthenticated users visiting them since they all shared the same cache key. This was awarded \$7500, making it my highest paid cache poisoning report.

GitLab CP-DoS

GitLab uses Google Cloud Platform and Fastly to host static files on `https://assets.gitlab-static.net/*`. Google Cloud Buckets support the use of the `x-http-method-override` header by default, which allows the HTTP method to be overridden. Appending the header `x-http-method-override: POST`, would return a 405 status code which Fastly does not cache by default. It was however possible to

send the header `x-http-method-override: HEAD` and poison the cache into returning an empty response body.

!

Moreover, the `PURGE` method was also enabled, drastically lowering the complexity of an attack. This was awarded a top-tier \$4,850 bounty. Besides GitLab, I was able to use the same technique on a multitude of other bounty targets.

X-Forwarded-Scheme - Rack Middleware

Ruby on Rails applications are often deployed alongside the Rack middleware. The Rack code below takes the value of the `x-forwarded-scheme` value and uses it as the scheme of the request.

!

Sending the `x-forwarded-scheme: http` header would result into a 301 redirect to the same location. If the response was cached by a CDN, it would cause a redirect loop, inherently denying access to the file. This was exploited on a good amount of bounty targets such as:

CP-DoS on Hackerone.com static files

Since Hackerone's cache configuration is set to only cache static files, cache poisoning attacks were restricted to static files.

!

Even though at the time of reporting DoS vulnerabilities were out of scope, this was still awarded a \$2500 bounty.

Single request DoS of www.shopify.com

The same technique also affected `www.shopify.com`, however Shopify's cache configuration increased the attack's impact. Since the server was not configured to cache HTML pages, but 301 requests were cached by default, it only took one untimed request to trigger the Cache Poisoning DoS.

!

This was initially awarded \$1300, however after further investigation this was discovered to also affect other localized subdomains and hosts such as `apps.shopify.com`. Since the vulnerability affected a number of Shopify hosts and only one request was required to poison the cache, the bounty amount was increased to \$6300.

Stored XSS on 21 subdomains

While testing a private program, I noticed the same vulnerability found on Hackerone affected all of their subdomains. This time however, the server also trusted the `X-forwarded-host` header on 301 redirects, allowing an attacker to redirect JS files to attacker controlled Javascript.

!

Since this could have lead to stored XSS on the target's main website and over 21 other subdomains, this was triaged as Critical and rewarded the maximum bounty of \$3000.

Cloudflare and Storage Buckets

With Cloudflare being the most widely-used content delivery network, Storage Buckets such as S3 are often times behind cloudflare. Unfortunately, this setup used to be vulnerable to cache poisoning by default.

Up until August 3rd 2021, Cloudflare used to cache 403 status codes even if there was no `Cache-control` directive. This made it possible to poison any file hosted on a S3 bucket and proxied through Cloudflare. Sending invalid Authorization headers would cause a cacheable 403 error.

S3 Bucket:

!

Azure Storage

Exodus uses the subdomain `downloads.exodus.com` to serve downloads such as the Exodus wallet installer. Since the files were stored on a Azure Storage Blob, it was possible to cause a cacheable 403 error with a crafted Authorization header. The Exodus team fixed the issue a few hours after receiving the report and awarded a \$2500 bounty.

!

Cloudflare also changed it's default configuration and now no longer caches 403 responses by default.

Fastly Host header injection

After reporting multiple cache poisoning vulnerabilities to the same bug bounty program, they agreed to sending me their Varnish Configuration file so I could more easily identify other inconsistencies. Upon skimming through the file, I found a snippet similar to the one below:

!

The snippet was used for a subdomain that was hosting map images. Requesting an image would look something like this:

!

The introduced rule made it that when the url path matched the regex, the cache key would only contain the coordinates extracted from the url and ignore all the other url components. Hence the image requested above would have the following cache key:

```
/4/151/16
```

Since the rule only included the extracted coordinates in the path, it meant I could send any host header to the backend and it would still match the same cache key. Unfortunately, that wasn't going to work since Fastly rejects any host header that is not whitelisted.

!

This mechanism was completely bypassed, by appending the header `Fastly-host` in the request. If the `fastly-host` header contained the whitelisted value, the host header could be changed to anything:

!

While it was possible to use the host header injection for CP-DoS, I was hoping to get more of it, so I decided to dig deeper. While looking at other Fastly hosts on the same program, I found a html file on `redacted-cdn.com` which was vulnerable to DOM XSS. Since this was under the `redacted-cdn.com` origin, the xss by itself had no impact.

I was able to escalate it by using the `fastly-host` header, after discovering the host header was being forwarded, but the `fastly-host` was being used to generate the cache key. The following request would therefore match the cache-key of:

```
https://assets.redacted.com/test.html
```

!

Since both hosts were behind the same loadbalancer, it was possible to cache files hosted on `redacted-cdn.com` under `assets.redacted.com`, inherently allowing me to move the vulnerable html file on a different domain and achieve xss under a different origin.

Injecting Keyed Parameters

Quite often, caches are configured to only include specific GET parameters in the cache key. This is especially common in CDNs hosting images which use parameters to modify the image size or format.

While testing a target using Fastly to cache images, I noticed the `size` parameter was included in the cache key, but all others were ignored. If two `size` parameters were added, both were included in the cache key, but the backend server used the value from the last parameter. Considering Fastly (Varnish), does not do any url normalization before generating the cache key, I was able to come up with the following DoS method:

!

URL encoding the second `size` parameter caused it to be ignored by the cache, but used by the backend. Giving the parameter a value of 0 would result in a cacheable 400 Bad Request.

User Agent Rules

Due to the high amount of traffic tools like FFUF or Nuclei generate, some developers decided to block requests matching their user-agents. Ironically, these tweaks can introduce unwanted cache poisoning DoS opportunities.

!

I found this worked on multiple targets, with user-agents from different tools or scanners.

Illegal Header Fields

The header name format is defined in [RFC7230](#) as follows:

!

In theory, if a header name contains characters other than the ones listed in **tchar** it should be rejected with a 400 Bad request. In practice however, servers don't always respect the RFC. The easiest way to exploit this nuance, was by targeting Akamai which doesn't reject invalid headers, but forwards them and caches any 400 error as long the cache-control header is not present.

!

Sending a header containing an illegal character, \ would cause a cacheable 400 Bad Request error. This was one of the most commonly identified patterns throughout my testing.

Finding new headers

Besides a few novel cases where attributes of the request-line could be used to poison the cache, the majority of Cache Poisoning vulnerabilities detected were caused by unkeyed headers.

Since I wanted to expand my list of headers, I used Google's BigQuery to query the HTTP Archive for values used in the **Vary** response header. The **Vary** header contains header names which should be keyed by the Cache server. This allowed me to find a few extra vulnerable instances, which wouldn't have been detected otherwise.

Here is the list of headers merged with Param-Miner's header list.

<https://gist.github.com/iustin24/92a5ba76ee436c85716f003dda8eccc6> (2917L)

Common Headers

The list below shows all the headers which were used to exploit over 70 caching servers.

!

Conclusion

Identifying Cache Poisoning vulnerabilities can be as easy as running a header brute force and detecting unkeyed headers, however limiting testing to that can often times miss subtle poisoning techniques laying in the complexity of server stacks. Custom caching configurations, differences in URL parsing or undocumented headers introduce unexpected behaviours which can result in cached arbitrary redirects, DoS or even overwriting of JS files.